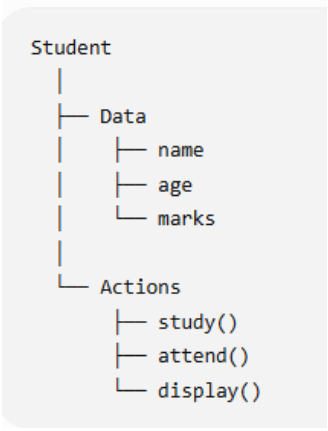
OOPs

OOPs stands for **Object-Oriented Programming System** (commonly called **Object-Oriented Programming**).

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects, which represent real-world entities. These objects encapsulate **data (attributes)** and **behavior (methods)**, making the code more modular and reusable.

It is a way of writing Python programs using **classes and objects**.
OOP is a programming approach where we organize data and functions together inside classes and create objects from those classes.

Think about a **Student**:



Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(self.name, self.age)


s1 = Student("Abhinav", 22)
s1.display()
```

Here:

- Student → **Class**
- s1 → **Object**
- name, age → **Data/Attributes**
- display() → **Method**

Why do we use OOP?

- **Organize** large programs
- **Reuse** code
- **Protect** data

- **Maintain** code easily
- Build complex applications more easily

Key Concepts of OOP in Python

1. **Class** – A blueprint for creating objects.
2. **Object** – An instance of a class.
3. **Encapsulation** – Hiding the internal details of an object and restricting direct access to some of its components.
4. **Abstraction** – Hiding complex implementation details and exposing only the necessary parts.
5. **Inheritance** – A mechanism that allows a new class to derive properties and behavior from an existing class.
6. **Polymorphism** – The ability to take multiple forms, e.g., methods with the same name but different implementations in different classes.

OOP follows four main principles:

1. **Encapsulation** – Hiding the internal state of an object and restricting access to it.
2. **Abstraction** – Hiding complex implementation details and exposing only the necessary parts.
3. **Inheritance** – Enabling new classes to derive properties and methods from existing ones.
4. **Polymorphism** – Allowing the same interface to be used for different underlying data types.

Difference Between OOP in Python and Other Languages

Feature	Python	Java	C++	C#
Syntax	Simple & Dynamic	Strict & Verbose	Complex	Similar to Java
Multiple Inheritance	✅ Yes	❌ No (only interfaces)	✅ Yes	❌ No (only interfaces)
Encapsulation	✅ Uses `_` and `__` (weak enforcement)	✅ Uses `private`, `protected`, `public`	✅ Strong encapsulation	✅ Strong encapsulation
Polymorphism	✅ Dynamic typing	✅ Strict typing	✅ Compile-time & runtime	✅ Strong polymorphism
Memory Management	✅ Automatic (Garbage Collection)	✅ Automatic (JVM)	❌ Manual	✅ Automatic
Performance	Slower (interpreted)	Faster (compiled)	Very fast	Faster than Python

Benefits of OOP

✅ **1. Code Reusability (DRY Principle)**

- Inheritance allows reusing existing code without rewriting it.
- Example: A Car class can be reused in multiple projects.

✅ **2. Modularity (Easier Code Organization)**

- Classes help break down large code into smaller, manageable parts.

- Example: A BankAccount class can have different methods like deposit() and withdraw().

✅ **3. Encapsulation (Data Security)**

- Prevents direct modification of sensitive data.
- Example: A private __balance attribute in a BankAccount class.

✅ **4. Scalability & Flexibility**

- Polymorphism allows the same method to work for different types.
- Example: A make_sound() method can work for both Dog and Cat classes.

✅ **5. Real-World Representation**

- Objects model real-world entities, making software design intuitive.
- Example: Car, Employee, Customer, etc.

Class

A **class is a blueprint or template used to create objects** in Python.

A **class** in Python is a blueprint or template for creating objects. It defines attributes (variables) and methods (functions) that the objects will have.

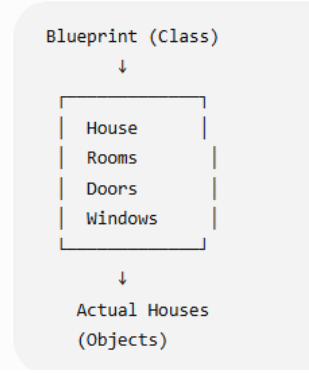
Syntax of a Class in Python

```
class ClassName:
    # Constructor (optional)
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2

    # Method
    def method_name(self):
        print("Method executed")
```

- class keyword is used to define a class.
- __init__() is a special method (constructor) that initializes object attributes.
- self represents the instance of the class.

Think about a **house blueprint** 🏠:



The **blueprint is not the actual house**. It describes how the house should be built.

Similarly:

Class = Blueprint

Object = Actual thing created from the blueprint

Example

```
class Student:
   name = "Abhinav"
   age = 22
```

Here, Student is a **class**.

We can create an object:

```
s1 = Student()

print(s1.name)
print(s1.age)
```

Output:

```
Abhinav
22
```

Class with a method

A class can contain both **data and functions**:

```
class Student:
   name = "Abhinav"

   def study(self):
      print("Student is studying")


s1 = Student()
print(s1.name)
s1.study()
```

Output:

```
Abhinav
Student is studying
```

Here study() is called a **method** because it is a function inside a class

Object

An object is a simply a collection of data(variable) and Methods(functions) that act on those data. Similarly, a class is a blueprint of an object.

An object (instance) is an instantiation of a class. When class is defined, only the description for the object is defined. Therefore, no memory or storage is allocated.

There is no memory allocation until we create its object. The Objector Instance Contains real Data or Information

Example:

```
class Abhi():
    def Output(self):
        print("Hello World")

vivo=Abhi()
vivo.Output()
```

Multiple objects

One class can create many objects:

```
class Student:
    pass

s1 = Student()
s2 = Student()
s3 = Student()
```

Real-world example

Class → Car

Objects → BMW, Audi, Toyota

The **Car class** describes what a car can have/do, while individual cars are **objects**.

Example-2:

```
class Kiran(): # class keyword class name
    a=10 # data member
    def Output(self): # method(self)
        print(self.a)
obj=Kiran()  # declaring of object
obj.Output() # using object we can call the methods
```

self Keyword:

We can acess the attribute and methods of the class (current class only)

Example:

```
class Abhi():
    def Output(self):
        print("Hello World")

vivo=Abhi()
vivo.Output()
```

Example-2:

```
class Abhi():
    a=10
    def show(self):
        print("this is class")
# obj name=class name()
```

```
sai=Abhi()
print(sai.a)
sai.show()
```

Example-3:

```
class Abhi():
   a=10 #attribute
   def display(self):
      print(self.a)

A1=Abhi()
A2=Abhi()
A1.display()
A2.display()
```

constructor

- Constructor is a special method that is Automatically called when you create an object.
- Constructors are generally used for instantiating an object.
- The task of constructors is to initialize(assign values)
- Constructor = Object's initial setup
- In Python the __init__() method is called the constructor and is always called when an object is created
- does'nt support multiple constructor

Example-1:

For example,
when you create a student, you may want to give the student a **name and age immediately**.

```
class Student:
   def __init__(self, name, age):
      self.name = name
      self.age = age


s1 = Student("Abhinav", 22)
print(s1.name)
print(s1.age)
```

Output:
Abhinav
22

Why do we use a constructor?

We use it to **initialize an object's data when the object is created**.
For example:
class Employee:

```
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary


e1 = Employee("Abhinav", 30000)
e2 = Employee("Rahul", 40000)
```

Now:
e1 → name = Abhinav, salary = 30000
e2 → name = Rahul, salary = 40000

Important terms
def __init__(self, name, age):

- __init__ → constructor/initializer method
- self → current object
- name, age → parameters
- self.name, self.age → object attributes

Example-2:

```
class Abhi():
    def __init__(self,a,b,c):
        self.A1=a
        self.B1=b
        self.C1=c

    def Output(self):
        print(self.A1, self.B1, self.C1)
p=Abhi(1,2,3)
p.Output()
```

Example-3:

```
class Mobiles():
    def __init__(self,Mobile_name,Mobile_Ram,Mobile_battery,Mobile_Price):
        self.a=Mobile_name
        self.c=Mobile_Ram
        self.d=Mobile_battery
        self.e=Mobile_Price

    def Mobile_Data(self):
        print("Mobile Name:",self.a)
        print("Mobile Ram:",self.c)
        print("Mobile Battery:",self.d)
        print("Mobile Price:",self.e)
```

```
name=input("Enter the Mobile Name:")
ram=input("Enter the Mobile Ram:")
bat=input("Enter the Mobile Battery:")
Price=float(input("Enter the Mobile Price:"))

Mobile_obj=Mobiles(name,ram,bat,Price)
Mobile_obj.Mobile_Data()
```

Example-4:

```
class Bikes():
    def __init__(self, bike_name, bike_cc, millage):
        self.a=bike_name
        self.b=bike_cc
        self.c=millage

    def Bike_Data(self):
        print("Bike Name:", self.a)
        print("Bike CC:", self.b)
        print("Bike Millage:", self.c)


bike_name = input("Enter Bike-1 Name")
bike_cc = input("Enter Bike-1 CC")
millage = input("Enter Bike-1 Millage")
Bike1 = Bikes(bike_name, bike_cc, millage)
Bike1.Bike_Data()


bike_name = input("Enter Bike-2 Name")
bike_cc = input("Enter Bike-2 CC")
millage = input("Enter Bike-2 Millage")

Bike2 = Bikes(bike_name, bike_cc, millage)
Bike2.Bike_Data()
```